Capítulo 8. Trabajar con datos

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *Los datos son algo muy valioso y durarán más que los propios sistemas.*
>
> —Tim Berners-Lee, informático

Cuando estás iniciándote en tu carrera como ingeniero de software , es fácil perderse en la idea de aprender lenguajes de programación, marcos de trabajo y herramientas. La realidad es que pasarás mucho tiempo en tu carrera trabajando con datos en diversas formas, y los datos constituyen la columna vertebral de todo lo que haces. Modelarás aplicaciones en torno a los datos, recopilándolos, almacenándolos, transformándolos y, en última instancia, descubriendo la mejor manera de transformarlos de nuevo para tus usuarios de una manera significativa.

Cuando piensas en datos, lo primero que te viene a la mente es el dato que almacenas en algún tipo de base de datos. Pero los datos son mu-

cho más que eso, incluyen comprender los diferentes tipos de datos, seleccionar las soluciones de almacenamiento adecuadas, diseñar modelos de datos eficientes, optimizar las consultas, garantizar la integridad de los datos y gestionar la evolución de los datos a lo largo del tiempo. Estas habilidades son tan importantes para tu éxito como desarrollador como tus habilidades fundamentales de codificación.

Aprender a trabajar bien con los datos puede ser un reto. Lo que funciona en el desarrollo puede fallar en la producción, desde bases de datos de clientes lentas hasta migraciones complejas y consultas ineficientes. Estos retos del mundo real son comunes en el desarrollo de software, y es importante estar preparado para ellos.

Este capítulo sienta las bases para trabajar con datos de forma eficaz y sirve de guía para convertirte en un desarrollador experto en datos. Aprenderás a identificar diferentes tipos de datos, seleccionar las herramientas adecuadas, explorar soluciones de almacenamiento eficientes, optimizar consultas y gestionar migraciones fluidas. Aunque no profundizarás en la administración de bases de datos ni en la ingeniería de datos, adquirirás las habilidades esenciales que todo desarrollador necesita para tomar decisiones informadas sobre los datos de tus aplicaciones y evitar los errores comunes que afectan a muchos desarrolladores.

Comprender los tipos y formatos de

datos

Cuando creas software, a menudo estás creando sistemas que procesan, transforman y presentan datos. Tanto si creas aplicaciones pequeñas como empresariales, el éxito de un proyecto puede estar ligado a sus datos y a cómo decidas estructurarlos, validarlos y procesarlos.

Cuando trabajes con tipos y formatos de datos, ten en cuenta estos factores:

Público

: ¿Quién leerá o procesará estos datos? ¿Personas o máquinas?

Requisitos de rendimiento

: ¿Es fundamental el tamaño o la velocidad de procesamiento?

Compatibilidad

: ¿Qué sistemas consumirán estos datos?

Complejidad

: ¿Qué grado de complejidad o variabilidad tiene tu estructura de datos?

Necesidades de validación

: ¿Necesitas validación de esquemas?

En esta sección, aprenderás conceptos para trabajar de forma eficaz con diferentes tipos de datos en tus aplicaciones. Explorarás la distinción fundamental entre datos estructurados y no estructurados, y comprenderás sus características, ventajas y limitaciones.

Datos estructurados frente a datos no estructurados

Los datos tienen dos tipos principales: estructurados y no estructurados. *Los datos estructurados* están organizados como una hoja de cálculo, con categorías y conexiones claras que facilitan su almacenamiento en bases de datos y su búsqueda. *Los datos no estructurados* incluyen texto normal, imágenes y documentos; estos necesitan herramientas especiales para trabajar con ellos. Comprender la distinción entre datos estructurados y no estructurados es importante para tomar decisiones de diseño eficaces en tus aplicaciones. Exploremos estos conceptos fundamentales.

Datos estructurados

Los datos estructurados se organizan según un modelo o esquema predefinido, con tipos de campos y relaciones coherentes. Piensa en ellos como información que puede encajar fácilmente en las filas y columnas de una hoja de cálculo o una base de datos relacional. En programación, esta estructura se representa a menudo mediante clases que definen el modelo de datos. En el siguiente ejemplo, una clase Java llamada `Customer` define las propiedades de un `Customer`:

```
public class Customer {
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private LocalDate birthDate;
```

```
    private Address address;

    // Getters and setters omitted for brevity
}
```

Las características clave de los datos estructurados incluyen las siguientes:

Formato coherente

: Todos los registros siguen el mismo esquema. Cada vez que crees un nuevo cliente en tu aplicación, este tendrá las propiedades definidas en la clase.

Relaciones bien definidas

: Conexiones claras entre entidades de datos.

Fácil de consultar

: Admite operaciones precisas de búsqueda y filtrado. En el caso del cliente, puedes encontrarlo por su nombre, apellido o una combinación de ambos.

Almacenamiento eficiente

: Optimizado para bases de datos con campos de longitud fija. El objeto cliente se asigna bien a las columnas de una tabla.

Los datos estructurados destacan en escenarios que requieren una integridad de datos estricta, consultas complejas y reglas de negocio bien establecidas. Algunos ejemplos son las transacciones financieras, la gestión de inventarios y los perfiles de usuario.

NOTA

Si trabajas con bases de datos relacionales como PostgreSQL o MySQL, lo más probable es que estés tratando con datos estructurados. Las definiciones de tablas y columnas en tu esquema dictan los campos existentes y el tipo de datos que cada uno puede contener.

Los datos estructurados suenan muy bien, ¿verdad? Lo son, pero también tienen sus limitaciones. Son relativamente inflexibles porque, si necesitas añadir o cambiar campos, a menudo es necesario modificar tanto el esquema de la base de datos como el código de la aplicación. Los datos estructurados también tienen dificultades para representar de forma eficiente información compleja o anidada.

NOTA SOBRE IA

Cuando necesites datos de prueba realistas que coincidan con tu esquema estructurado, las herramientas de IA pueden ser útiles. En lugar de crear manualmente docenas de registros de clientes falsos o buscar los datos adecuados o la biblioteca de generación, simplemente puedes pedir: «Genera 20 registros de clientes realistas con nombres, correos electrónicos, direcciones y fechas de nacimiento en formato JSON». La IA te permite ser específico sobre lo que quieres, como «Generar datos de productos de comercio electrónico para una librería vintage» o «Crear perfiles de usuario para una aplicación de fitness con distintos niveles de actividad». Esto resulta especialmente útil cuando estás aprendiendo o creando prototipos, ya que puedes obtener rápidamente datos significativos que hacen que tus aplicaciones parezcan reales.

Datos no estructurados

Los datos no estructurados no tienen un modelo de datos predefinido. No encajan perfectamente en filas y columnas. Esto incluye texto de formato libre como correos electrónicos, publicaciones en redes sociales y transcripciones de chats. Otros ejemplos de datos no estructurados son imágenes, audio, vídeo, registros con formatos inconsistentes y documentos (PDF y Word) que no tienen una estructura claramente definida.

Cuando trabajas con datos no estructurados, generalmente necesitas adoptar un enfoque diferente para procesarlos debido a su naturaleza sin procesar. Esto puede implicar un procesamiento del lenguaje natural (NLP) e e para el análisis de texto o la visión por computadora para imágenes y audio.

En el siguiente ejemplo, esta función toma algunos comentarios sin procesar de los clientes y utiliza el PLN para determinar si los comentarios son positivos o negativos, identifica palabras clave o temas y, a continuación, guarda esa información junto con los comentarios originales:

```
// Processing unstructured data often requires different approaches
public void processCustomerFeedback(String feedbackText) {
    // Extract sentiment using natural language processing
    Sentiment sentiment = nlpService.analyzeSentiment(feedbackText);

    // Identify key topics or keywords
    List<String> keywords = nlpService.extractKeywords(feedbackText);
```

```
    // Store both the original unstructured text and extracted insights
    feedbackRepository.save(new Feedback(feedbackText, sentiment, keywords));
}
```

Las características que definen los datos no estructurados incluyen las siguientes:

Formato variable

: Sin esquema ni estructura coherentes

Contenido rico

: Contiene información matizada, como emociones, opiniones o detalles visuales

Difícil de consultar de forma tradicional

: Requiere técnicas especializadas, como la búsqueda de texto completo

Dificultades de almacenamiento

: A menudo requiere sistemas especializados optimizados para objetos de gran tamaño

Los datos no estructurados son muy valiosos para aplicaciones que tratan con comunicación humana, contenido creativo o información compleja del mundo real. Trabajar con datos no estructurados suele requerir herramientas diferentes a las de los datos estructurados. En lugar de consultas SQL, puedes utilizar motores de búsqueda de texto completo como Elasticsearch, bibliotecas de PLN o lagos de datos especializados diseñados para un almacenamiento flexible.

Formatos de datos comunes

Como ingeniero de software, te encontrarás con varios formatos de datos a lo largo de tu carrera. Tu trabajo consiste en comprender los formatos, sus puntos fuertes y débiles, y cuándo utilizar uno u otro. A veces puedes elegir el formato adecuado, y otras veces esa decisión ya está tomada. En cualquier caso, comprenderlos te ayudará a tomar mejores decisiones de diseño y a trabajar de forma más eficaz en tus aplicaciones. Exploremos algunos de los formatos de datos más comunes con los que te encontrarás a lo largo de tu trayectoria.

JSON

JavaScript Object Notation (JSON) se ha convertido en el estándar de facto para el intercambio de datos en las aplicaciones modernas debido a su legibilidad y simplicidad. Aunque el nombre sugiere que está hecho para JavaScript, es independiente del lenguaje y admite tipos como objetos, matrices, cadenas, números, booleanos y valores nulos. El siguiente es un ejemplo de un objeto JSON que contiene información sobre un usuario:

```
{
  "id": 1001,
  "firstName": "Jane",
  "lastName": "Smith",
  "email": "jane.smith@example.com",
  "birthDate": "1985-03-15",
  "address": {
    "street": "123 Main Street",
    "city": "Boston",
```

```
      "state": "MA",
      "zipCode": "02108",
      "country": "USA"
    }
  }
```

JSON es una opción realmente excelente para API web, archivos de configuración y bases de datos de documentos. Entre sus limitaciones se incluyen la falta de compatibilidad con comentarios, formatos de fecha o datos binarios sin codificación. Para la mayoría de las aplicaciones modernas, especialmente aquellas con interfaces de JavaScript, JSON es la opción natural.

XML

XML, o lenguaje de marcado extensible, es más prolijo que JSON, pero ofrece una validación robusta a través de la definición de esquema XML (XSD) y la definición de tipo de documento (DTD). XSD proporciona capacidades de definición de esquemas, lo que te permite especificar tipos de datos, restricciones y reglas estructurales para tus documentos XML. También puedes utilizar expresiones XPath para la selección y transformación precisa de datos (XSLT) para convertir XML a otros formatos dentro de tus documentos XML.

XML admite espacios de nombres, que ayudan a evitar conflictos de nombres en documentos complejos al proporcionar identificadores únicos para elementos de diferentes vocabularios o esquemas XML. Para procesar documentos XML realmente grandes, hay disponibles múltiples analizadores especializados que pueden mejorar significati-

vamente el rendimiento mediante streaming, análisis basado en eventos o técnicas eficientes en cuanto a memoria. A continuación se muestra un ejemplo de un documento XML que contiene información sobre un usuario:

```
<?xml version="1.0" encoding="UTF-8"?>
<customer>
    <id>1001</id>
    <firstName>Jane</firstName>
    <lastName>Smith</lastName>
    <email>jane.smith@example.com</email>
    <birthDate>1985-03-15</birthDate>
    <address>
        <street>123 Main Street</street>
        <city>Boston</city>
        <state>MA</state>
        <zipCode>02108</zipCode>
        <country>USA</country>
    </address>
</customer>
```

XML sigue siendo muy utilizado en sistemas empresariales, formatos de documentos (como DOCX, SVG) y archivos de configuración para aplicaciones complejas. Elige XML cuando la validación de esquemas, los espacios de nombres o la compatibilidad con sistemas orientados a documentos sean importantes.

CSV

Los valores separados por comas (CSV) almacenan datos tabulares como texto sin formato, utilizando comas para separar los valores y saltos de línea para separar los registros. La primera fila suele contener los encabezados de las columnas. Aunque el formato CSV es sencillo, carece de definiciones de tipos de datos integradas y de reglas coherentes para el manejo de caracteres especiales. El siguiente ejemplo contiene columnas con información del usuario y una fila con datos de muestra:

```
id,firstName,lastName,email,birthDate,address.street,address.city,address.state,
address.zipCode,address.country
1001,Jane,Smith,jane.smith@example.com,1985-03-15,"123 Main Street",
Boston,MA,02108,USA
```

El formato CSV funciona bien para la exportación de datos, el intercambio de datos sencillos y la compatibilidad con aplicaciones de hojas de cálculo. Úsalo para datos tabulares con una estructura sencilla, pero ten cuidado con los caracteres internacionales, las comas dentro de los campos y los tipos de datos complejos.

NOTA

Cuando trabajes con CSV, ten siempre en cuenta cómo gestionar los casos especiales: campos vacíos, campos que contienen comas o comillas y saltos de línea dentro de los campos. La mayoría de las bibliotecas CSV ofrecen opciones para estos casos, pero debes configurarlas explícitamente.

YAML

YAML Ain't Markup Language (YAML) ofrece una sintaxis más fácil de usar que JSON o XML, con soporte para comentarios, referencias y texto multilínea. El uso de indentación define la estructura, lo que mejora la legibilidad, pero requiere prestar mucha atención al formato, ya que incluso un solo espacio mal colocado puede provocar errores de análisis. Las buenas prácticas incluyen el uso de una indentación coherente (normalmente dos espacios), la validación de los archivos YAML antes de su implementación y el uso de un editor compatible con YAML para detectar a tiempo los problemas de formato. A continuación se muestra un ejemplo de un usuario definido en formato YAML:

```
customer:
  id: 1001
  firstName: Jane
  lastName: Smith
  email: jane.smith@example.com
  birthDate: 1985-03-15
  address:
    street: 123 Main Street
    city: Boston
    state: MA
    zipCode: 02108
    country: USA
```

YAML es ideal para archivos de configuración, especialmente en herramientas de DevOps como Docker Compose, Kubernetes y canalizaciones CI/CD. Su legibilidad lo hace excelente para archivos editados por

personas, aunque debes tener cuidado con la indentación y los caracteres especiales.

La mayoría de las aplicaciones utilizan múltiples formatos, como JSON para las API, YAML para la configuración y CSV para descargar los datos de los clientes. La elección correcta depende de tus requisitos y limitaciones específicos.

Consideraciones sobre datos especializados

Más allá de los formatos comunes que aprendiste en la sección anterior, encontrarás tipos de datos especializados que son menos comunes, pero que siguen siendo importantes de comprender y requieren un enfoque particular para manejarlos.

Datos binarios

Los datos binarios abarcan todo, desde imágenes y documentos hasta archivos de audio y contenido cifrado. A diferencia de los formatos basados en texto, los datos binarios no se pueden leer ni manipular directamente sin una codificación y decodificación adecuadas. El siguiente ejemplo muestra cómo leer una imagen y convertirla en datos binarios, en este caso una matriz de bytes:

```
// Reading an image file as binary data
try (FileInputStream fis = new FileInputStream("dogs-playing-poker.png");
     ByteArrayOutputStream bos = new ByteArrayOutputStream()) {

    byte[] buffer = new byte[1024];
```

```
        int bytesRead;
        while ((bytesRead = fis.read(buffer)) != -1) {
            bos.write(buffer, 0, bytesRead);
        }

        byte[] imageBytes = bos.toByteArray();
        // Now you can work with the binary data
    }
```

Una vez que hayas cargado un archivo de imagen en la memoria como una matriz de bytes, puedes hacer muchas cosas con él:

- Mostrar la imagen en una aplicación GUI
- Procesar o manipular la imagen (cambiar el tamaño, recortar, aplicar filtros, etc.)
- Convertirla a un formato de imagen diferente (PNG a JPEG, etc.)
- Subirla a un servidor o almacenamiento en la nube
- Enviarla como parte de una solicitud HTTP
- Incrustarla en un documento PDF
- Extraer metadatos de la imagen.

Cuando trabajes con datos binarios, hay varias prácticas clave que debes seguir. Utiliza bibliotecas específicas diseñadas para formatos de archivo concretos. La codificación Base64 resulta útil cuando es necesario incluir datos binarios en formatos de texto. La representación hexadecimal proporciona una forma legible de examinar bytes individuales durante la depuración y el análisis. Los archivos binarios de gran tamaño requieren una gestión cuidadosa de la memoria. Imple-

menta siempre el control de errores para detectar archivos de datos corruptos o incompletos.

Datos de fecha y hora

La comunidad de ingeniería de software suele bromear diciendo que los problemas más difíciles de la informática son nombrar las cosas y la invalidación de la caché (véase «Naming Things Is Hard»). El manejo de la fecha y la hora merece un lugar cerca de la cima de esa lista debido a su engañosa complejidad. Trabajar con datos temporales presenta numerosos retos derivados de las zonas horarias, los ajustes del horario de verano, los diversos sistemas de calendario y las conversiones de formato.

En la siguiente aplicación Java, tomamos una fecha y hora locales, las asociamos a una zona horaria específica y, a continuación, añadimos el tiempo de viaje antes de convertirlas a otra zona horaria. Por último, formateamos el resultado `ZonedDateTime` para su visualización, lo que nos permite presentar de forma clara la hora de llegada a la región de destino:

```
// Reading an image file as binary data
try (FileInputStream fis = new FileInputStream("dogs-playing-poker.png");
     ByteArrayOutputStream bos = new ByteArrayOutputStream()) {

    byte[] buffer = new byte[1024];
    int bytesRead;
    while ((bytesRead = fis.read(buffer)) != -1) {
        bos.write(buffer, 0, bytesRead);
```

```
        }

        byte[] imageBytes = bos.toByteArray();
        // Now you can work with the binary data
    }
```

A continuación se indican algunas consideraciones clave para trabajar con datos de fecha y hora:

- Almacenar siempre las fechas internamente en formato UTC ( ), convirtiéndolas a las zonas horarias locales solo para su visualización.
- Utiliza el formato ISO 8601 para el intercambio de fecha y hora entre sistemas
- Aprovecha las bibliotecas modernas de fecha/hora en lugar de crear soluciones personalizadas
- Sé explícito sobre las zonas horarias en las interfaces de usuario para evitar confusiones e es

Grandes conjuntos de datos

A medida que las aplicaciones se amplían, también lo hacen los datos con los que trabajas. Lo que antes era un sistema sencillo que utilizaba tipos de datos básicos, ahora contiene conjuntos de datos muy grandes que son demasiado voluminosos para procesarlos en la memoria de una sola vez, lo que requiere un enfoque especializado. En el siguiente ejemplo, mostramos cómo procesar archivos CSV grandes utilizando un enfoque de streaming que lee y procesa los registros uno por uno:

```
// Stream processing approach for large CSV files
try (Reader reader = new FileReader("massive-data.csv");
CSVParser parser = CSVFormat.DEFAULT.withHeader().parse(reader)) {

parser.forEach(record -> {
   // Process one record at a time
   // without loading everything into memory
   processRecord(record);

   });
}
```

Al manejar conjuntos de datos grandes, ten en cuenta lo siguiente:

- Utiliza enfoques de streaming que procesen los datos de forma incremental
- Implementa la paginación para las respuestas de la API y las interfaces de usuario.
- Considera optimizaciones de bases de datos como la indexación y el ajuste de consultas
- Evalúa tecnologías especializadas, como almacenes de datos o marcos de procesamiento distribuido, para conjuntos de datos extremadamente grandes.
- Implementa un control de concurrencia adecuado mediante el uso de bloqueos, operaciones atómicas o estructuras de datos inmutables para evitar conflictos de datos cuando varios subprocesos acceden simultáneamente a datos compartidos.
- Realiza pruebas con volúmenes de datos realistas en las primeras fases del desarrollo

Estas consideraciones especializadas cobran cada vez más importancia a medida que tus aplicaciones crecen en complejidad y escala. Aunque no es necesario que seas un experto en todas estas áreas de inmediato, conocer estas consideraciones te ayudará a reconocer cuándo los enfoques estándar pueden no ser suficientes y cuándo debes buscar soluciones o conocimientos más especializados.

En esta sección, has aprendido sobre diferentes tipos de datos, como los datos estructurados y los no estructurados. También has visto algunos tipos de datos comunes y especializados con los que trabajarás a lo largo de tu carrera. Ahora que tienes una visión general de los tipos de datos, veremos cómo almacenar esos datos de forma eficaz en la próxima sección .

Almacenamiento eficaz de datos

Una vez que hayas determinado las estructuras de datos con las que trabajas, debes elegir el mecanismo de almacenamiento adecuado para tus datos. Elegir el almacenamiento adecuado desde el principio puede evitar problemas más adelante. En esta sección, aprenderás a seleccionar la base de datos adecuada para las necesidades de tu aplicación y comprenderás las ventajas y limitaciones de los diferentes tipos de bases de datos. Estos fundamentos te ayudarán a tomar decisiones bien informadas que afectarán al rendimiento, la escalabilidad y la facilidad de mantenimiento de tu aplicación.

Tipos de bases de datos y sus casos de uso

Seleccionar una base de datos para tu próxima aplicación no consiste en seguir las tendencias de las redes sociales o copiar las preferencias de tus colegas. En su lugar, toma una decisión informada basada en los requisitos específicos de tu aplicación. A la hora de elegir una solución de almacenamiento, ten en cuenta estos factores clave:

Complejidad de la estructura de datos

: ¿Qué grado de complejidad tienen las relaciones en tus datos?

Patrones de lectura frente a patrones de escritura

: ¿Tu aplicación realizará más operaciones de lectura o de escritura?

Complejidad de las consultas

: ¿Qué tipo de preguntas realizarás a tus datos?

Necesidades de escalabilidad

: ¿Cuánto crecerán tus datos con el tiempo?

Requisitos de coherencia

: ¿Qué importancia tiene la coherencia de los datos para tu aplicación?

Examinemos dos aplicaciones con diferentes requisitos de base de datos. En primer lugar, considera una aplicación de blog para compartir tus conocimientos y tu pasión. Comenzarás por definir las entidades (usuarios, publicaciones, etiquetas, comentarios, etc.) y sus relaciones: los usuarios escriben muchas publicaciones y las publicaciones tienen

muchas etiquetas y comentarios. Esta estructura claramente definida sugiere que una base de datos relacional sería la elección adecuada.

En segundo lugar, imagina que creas un catálogo de productos para mercancía personalizada. En este caso, tus productos no siguen una estructura uniforme y tienen atributos variables que pueden cambiar con frecuencia. Para este escenario, probablemente sería más adecuada una base de datos de documentos.

NOTA

Las opciones de bases de datos pueden evolucionar con tu aplicación, y ninguna elección es permanente. Muchos proyectos exitosos comienzan de forma sencilla y migran a soluciones más complejas a medida que cambian las necesidades. No te sientas presionado para elegir la solución «perfecta» ni para diseñar en exceso desde el primer día.

Ahora, exploremos los tipos de bases de datos y cuándo podrías elegir cada una de ellas.

Bases de datos relacionales

L*as bases de datos relacionales* organizan los datos en tablas con filas y columnas, estableciendo relaciones entre las tablas mediante claves. Se basan en sólidos fundamentos matemáticos (álgebra relacional) y garantizan la integridad de los datos a través de las propiedades ACID:

Atomicidad

Las transacciones son operaciones de todo o nada. O bien todos los cambios de una transacción se completan correctamente, o bien ninguno de ellos lo hace. Si alguna parte falla, se revierte toda la transacción para mantener la coherencia de los datos.

Consistencia

La base de datos permanece en un estado válido antes y después de cada transacción. Se conservan todas las reglas de integridad de los datos, las restricciones y las relaciones, lo que garantiza que la base de datos nunca entre en un estado no válido.

Aislamiento

Las transacciones simultáneas no interfieren entre sí. Cada transacción parece ejecutarse de forma aislada, incluso cuando se ejecutan varias transacciones simultáneamente, lo que evita problemas como lecturas sucias o actualizaciones perdidas.

Durabilidad

Una vez que se confirma una transacción, sus cambios se almacenan de forma permanente y sobreviven a los fallos del sistema. Los datos persisten incluso si la base de datos se bloquea o se produce un corte de energía inmediatamente después de la confirmación.

Utiliza bases de datos relacionales en estos casos:

- Tus datos tienen relaciones claras entre entidades.
- Necesitas consultas y uniones complejas.
- Las transacciones y la coherencia de los datos son fundamentales.
- Tienes datos estructurados que cambian con poca frecuencia.

Algunos ejemplos populares son PostgreSQL, MySQL, Oracle y SQL Server.

A continuación se muestra un ejemplo sencillo de creación de tablas en una base de datos relacional en Java:

```
// Using JDBC to create tables
String createUserTable = """
            CREATE TABLE users (
                id SERIAL PRIMARY KEY,
                username VARCHAR(50) UNIQUE NOT NULL,
                email VARCHAR(100) UNIQUE NOT NULL,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """;

String createPostTable = """
            CREATE TABLE posts (
                id SERIAL PRIMARY KEY,
                user_id INTEGER REFERENCES users(id),
                title VARCHAR(200) NOT NULL,
                content TEXT NOT NULL,
                published_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """;

try (Connection conn = dataSource.getConnection();
     Statement stmt = conn.createStatement()) {
   stmt.executeUpdate(createUserTable);
   stmt.executeUpdate(createPostTable);

   // Insert a user
```

```
    String insertUser = "INSERT INTO users (username, email) VALUES (?, ?)";
    try (PreparedStatement pstmt = conn.prepareStatement(insertUser)) {
        pstmt.setString(1, "john_doe");
        pstmt.setString(2, "john@example.com");
        pstmt.executeUpdate();
    }
}
```

Bases de datos de documentos

Las bases de datos de documentos almacenan los datos en documentos flexibles, similares a JSON, en lugar de en tablas rígidas. Cada documento puede tener una estructura diferente, lo que permite una mayor flexibilidad que las bases de datos relacionales.

Utiliza bases de datos de documentos en estos casos:

- Tu estructura de datos varía entre registros.
- Necesitas escalabilidad horizontal para grandes conjuntos de datos.
- Los requisitos de tu aplicación cambian con frecuencia.
- Estás creando sistemas de gestión de contenidos o catálogos.

Algunos ejemplos populares son MongoDB, Couchbase y Firebase Firestore.

A continuación se muestra cómo se puede almacenar una entrada de blog en una base de datos de documentos:

```
// Using MongoDB Java driver
Document post = new Document()
    .append("title", "Getting Started with MongoDB")
    .append("author", new Document("name", "Jane Developer")
    .append("email", "jane@example.com"))
    .append("tags", Arrays.asList("database", "nosql", "mongodb"))
    .append("comments", Arrays.asList(
        new Document("user", "reader1").append("text", "Great article!"),
        new Document("user", "reader2").append("text", "Thanks for sharing!")
    ))
    .append("publishedAt", new Date())
    .append("createdAt", new Date());

try {
    collection.insertOne(post);
} catch (MongoException e) {
    log.error("Error: " + e.getMessage());
}
```

Almacenes de clave-valor

Los almacenes de clave-valor son la forma más simple de bases de datos NoSQL, ya que almacenan datos como una colección de pares clave-valor. Son extremadamente rápidos para operaciones simples, pero tienen capacidades de consulta limitadas.

Utiliza almacenes clave-valor en estos casos:

- Necesitas un rendimiento de lectura/escritura ultrarrápido.

- Tus patrones de acceso a los datos son sencillos (principalmente por clave).
- Estás creando capas de almacenamiento en caché, almacenes de sesiones o preferencias.
- Tu caso de uso valora la velocidad por encima de las consultas complejas.

Algunos ejemplos populares son Redis, Amazon DynamoDB y Riak.

A continuación, se muestra un ejemplo del uso de un almacén de clave-valor para la gestión de sesiones:

```
// Using Jedis (Redis Java client)
try (Jedis jedis = jedisPool.getResource()) {
    // Store session with 30-minute expiration
    String sessionId = generateSessionId();
    jedis.setex("session:" + sessionId, 1800, userJson);

    // Retrieve session
    String storedSession = jedis.get("session:" + sessionId);
}
```

Bases de datos de grafos

Las bases de datos de grafos están diseñadas para almacenar y consultar de manera eficiente datos altamente interconectados, representando la información como nodos (entidades) y aristas (relaciones entre esas entidades). A diferencia de las bases de datos relacionales que utilizan tablas y claves externas, las bases de datos de grafos convierten

las relaciones en elementos fundamentales del modelo de datos. Las bases de datos de grafos utilizan lenguajes de consulta comunes que facilitan la búsqueda de patrones y el seguimiento de las conexiones en tus datos.

Utiliza bases de datos gráficas en estos casos:

- Tus datos están muy interconectados.
- Las relaciones son tan importantes como los datos en sí.
- Necesitas realizar recorridos complejos o búsquedas de rutas.
- Estás creando redes sociales, motores de recomendación o grafos de conocimiento.

Algunos ejemplos populares son Neo4j, JanusGraph y Amazon Neptune.

A continuación se muestra un ejemplo práctico en Neo4j utilizando Java, que muestra cómo crear un nodo de persona con propiedades y establecer una relación de «seguimiento» entre dos usuarios:

```
// Using Neo4j Java driver
try (Session session = driver.session()) {
    session.writeTransaction(tx -> {
       tx.run("CREATE (user:Person {name: $name, email: $email})",
              parameters("name", "Alice", "email", "alice@example.com"));
       tx.run("MATCH (a:Person {name: $person1}), (b:Person {name: $person2}) " +
              "CREATE (a)-[:FOLLOWS]->(b)",
              parameters("person1", "Alice", "person2", "Bob"));
       return null;
```

```
    });
}
```

Las bases de datos gráficas son muy útiles para revelar patrones y conexiones ocultos que serían difíciles de descubrir en las bases de datos relacionales tradicionales. Utilizan un modelo sencillo y son ideales para encontrar conexiones en tus datos. Son la mejor herramienta cuando tu aplicación necesita comprender y utilizar las relaciones entre los distintos elementos de información.

Bases de datos vectoriales

Las bases de datos vectoriales son sistemas de bases de datos especializados diseñados para manejar vectores numéricos de alta dimensión y realizar búsquedas de similitud de manera eficiente. A diferencia de las bases de datos tradicionales que almacenan valores discretos como texto o números, las bases de datos vectoriales almacenan representaciones matemáticas de datos (incrustaciones) que capturan el significado semántico y permiten aplicaciones impulsadas por IA.

Cuando se trata de bases de datos vectoriales, hay dos conceptos clave que debes comprender: las incrustaciones vectoriales y la búsqueda por similitud.

L*as incrustaciones vectoriales* son una forma de almacenar datos. Tienes matrices de números (normalmente de cientos a miles de dimensiones) que representan puntos de datos en un espacio de alta dimensión. Los elementos similares tienen vectores que están muy próximos

entre sí en este espacio, mientras que los elementos diferentes están muy separados.

La búsqueda por similitud es un a forma de acceder a los datos. Es la operación principal: encontrar los elementos más similares a lo que estás buscando. Piensa en ello como si preguntaras «Muéstrame cosas que sean como esto» en lugar de «Muéstrame cosas que coincidan exactamente con esto».

Las bases de datos vectoriales impulsan muchas aplicaciones modernas de IA, y tú podrías utilizarlas en los siguientes escenarios:

Búsqueda semántica

: Búsqueda de documentos basada en el significado en lugar de en palabras clave.

Sistemas de recomendación

: Sugerir productos, contenidos o usuarios similares.

Generación aumentada por recuperación (RAG)

: Proporcionar contexto relevante a modelos de lenguaje grandes (LLMs).

Búsqueda de imágenes/vídeos

: Búsqueda de medios visualmente similares

Detección de anomalías

: Identificar valores atípicos en datos de alta dimensión

Deduplicación

: Búsqueda de contenido casi duplicado

Algunos ejemplos populares son: Pinecone, Weaviate, Chroma y Qdrant.

Para seleccionar la base de datos adecuada, es necesario comprender la forma de tus datos y los requisitos de tu aplicación. Las bases de datos relacionales ofrecen estructura y coherencia para relaciones bien definidas. Las bases de datos de documentos proporcionan flexibilidad para esquemas en evolución. Los almacenes de clave-valor ofrecen velocidad para patrones de acceso a datos sencillos. Las bases de datos gráficas destacan en la gestión de relaciones complejas. Las bases de datos vectoriales impulsan muchas aplicaciones modernas de IA.

A la hora de elegir una base de datos, recuerda que no es una decisión que vaya a determinar el éxito o el fracaso de tu carrera. Aunque tener millones de usuarios puede requerir cambiar de base de datos más adelante, ¡en realidad eso es una señal de éxito! La clave es elegir en función de tus necesidades actuales y seguir avanzando.

Persistencia y gestión de datos

Ahora que ya conoces los tipos de bases de datos disponibles, es el momento de explorar cómo trabajar con ellas de forma eficaz. Independientemente del lenguaje o el marco que utilices, es probable que existan diferentes niveles de abstracción. Al igual que al elegir la base de datos adecuada, debes decidir qué nivel de abstracción tiene sentido para la aplicación en la que estás trabajando.

En este escenario, la dinámica del equipo también juega un papel importante en la toma de decisiones. Si tienes un equipo grande de

desarrolladores con experiencia en SQL, pero sin experiencia en mapeadores relacionales de objetos (ORM) como Hibernate, tal vez no tenga mucho sentido introducir la API de persistencia de Java en tu proyecto.

En esta sección, aprenderás sobre diferentes patrones de persistencia, gestión de conexiones, modelos de consistencia y planificación para el crecimiento de los datos. La mayoría de los ejemplos aquí están en Java, pero si ese no es tu lenguaje, no te preocupes. En su lugar, céntrate en los patrones y el nivel de abstracción y en cómo podrían aplicarse a tu lenguaje o marco de trabajo favorito.

Veamos algunas opciones para los patrones de persistencia. El nivel de abstracción que elijas para la persistencia de datos puede afectar significativamente a tu velocidad de desarrollo, la mantenibilidad del código y el rendimiento de la aplicación. No existe una solución única para todos los casos. Tu elección debe estar influenciada por la complejidad de tu aplicación, la experiencia de tu equipo y el calendario de tu proyecto.

Acceso directo a la base de datos

El acceso directo a la base de datos es el nivel más bajo de abstracción, en el que se escribe SQL sin procesar o lenguajes de consulta específicos de la base de datos. Este enfoque te ofrece un control total, pero requiere un profundo conocimiento de las bases de datos.

En el siguiente ejemplo, la consulta selecciona las columnas necesarias de la tabla de usuarios, ejecuta esa consulta y, a continuación, itera so-

bre el conjunto de resultados:

```
// Direct JDBC access example
String query = "SELECT id, username, email FROM users WHERE active = true " +
            "ORDER BY created_at DESC";
try (Connection conn = dataSource.getConnection();
     PreparedStatement stmt = conn.prepareStatement(query);
     ResultSet rs = stmt.executeQuery()) {

    while (rs.next()) {
        User user = new User(rs.getLong("id"),
                             rs.getString("username"),
                             rs.getString("email"));
        users.add(user);
    }
}
```

Patrón de repositorio

El patrón de repositorio proporciona una capa de abstracción entre la lógica de negocio y el código de acceso a los datos. Actúa como una interfaz similar a una colección para acceder a los objetos de dominio, ocultando la complejidad de las operaciones de la base de datos. Este patrón ofrece varias ventajas clave:

Separación de preocupaciones

Aísla la lógica de acceso a los datos de la lógica empresarial

Pruebas

Facilita las pruebas unitarias al permitir repositorios simulados

Organización del código

Centraliza la lógica de acceso a los datos en un solo lugar

`UserRepository` Si observas el siguiente ejemplo de código, podrás ver cómo el repositorio implementa estos principios:

1. El repositorio encapsula todas las consultas SQL y las operaciones de la base de datos.
2. La abstracción de la base de datos protege a la aplicación de los cambios en el esquema de la base de datos y los detalles de implementación.
3. Proporciona una interfaz limpia y centrada en el dominio (`findActiveUsers`).
4. El manejo de errores está estandarizado a través de `RepositoryException` .
5. La gestión de conexiones se maneja adecuadamente con `try` -with-resources.

```
// Repository pattern example
public class UserRepository {
    private final DataSource dataSource;

    public UserRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public List<User> findActiveUsers() {
        List<User> users = new ArrayList<>();
        String query = "SELECT id, username, email FROM users WHERE " +
                "active = true ORDER BY created_at DE";
```

```
        try (Connection conn = dataSource.getConnection();
             PreparedStatement stmt = conn.prepareStatement(query);
             ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                User user = new User(rs.getLong("id"),
                                     rs.getString("username"),
                                     rs.getString("email"));
                users.add(user);
            }
        } catch (SQLException e) {
            throw new RepositoryException("Failed to find active users", e);
        }

        return users;
    }
}
```

Mapeo relacional de objetos

Los marcos*de mapeo relacional de objetos* (*ORM*) como Hibernate, JPA y Entity Framework proporcionan el más alto nivel de abstracción en la interacción con bases de datos. Estas herramientas tienden un puente entre la programación orientada a objetos y las bases de datos relacionales al mapear objetos de dominio directamente a tablas de bases de datos, eliminando la necesidad de escribir SQL manualmente y mapear resultados en la mayoría de los casos.

El siguiente código muestra cómo las anotaciones JPA definen una entidad Usuario con metadatos de mapeo de bases de datos. El repositorio

utiliza un lenguaje de consulta simplificado en lugar de SQL sin procesar para recuperar los usuarios activos ordenados por fecha de creación:

```
// JPA/Hibernate example
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;
    private String email;
    private boolean active;
    private LocalDateTime createdAt;

    // Getters and setters...
}

// Using the entity with JPA
@Repository
public class UserJpaRepository {
    @PersistenceContext
    private EntityManager entityManager;

    public List<User>  findActiveUsers() {
        return entityManager.createQuery(
                "SELECT u FROM User u WHERE u.active = true " +
                        "ORDER BY u.createdAt DESC",User.class)
                .getResultList();
    }
}
```

NOTA

Un error común que cometen los desarrolladores noveles es asumir que una mayor abstracción (como los ORM) siempre es mejor. A veces, el SQL directo ofrece un mejor rendimiento y un mayor control. Los ORM pueden facilitar el inicio, pero a menudo son más difíciles de depurar. Aprende a evaluar las ventajas e inconvenientes para tus necesidades específicas en lugar de seguir sin pensar un único enfoque.

Conexiones y transacciones de bases de datos

Una vez que hayas tomado una decisión sobre la base de datos que vas a utilizar y cómo vas a interactuar con ella, hay otras dos características que pueden mejorar el rendimiento y la fiabilidad de tu aplicación. Las conexiones a la base de datos definen cómo se conectarán tus aplicaciones a la base de datos, y las transacciones definen cómo funcionarán las múltiples operaciones en una sola unidad de trabajo.

Conexiones a bases de datos

Cuando aprendiste por primera vez sobre bases de datos y cómo conectarte a ellas, probablemente escribiste código que permitía una única conexión. Cuando empiezas a trabajar con aplicaciones del mundo real, comprender la gestión adecuada de las conexiones es fundamental para el rendimiento y la fiabilidad. La mayoría de las aplicaciones utilizan un grupo de conexiones e para reutilizar las conexiones a la base de datos en lugar de crear una nueva para cada operación.[1]

El siguiente ejemplo muestra cómo utilizar el grupo de conexiones Hikari (HikariCP) en Java para conectarse a una base de datos PostgreSQL local:

```
// HikariCP connection pool configuration
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:postgresql://localhost:5432/myapp");
config.setUsername("user");
config.setPassword("password");
config.setMaximumPoolSize(10);

HikariDataSource dataSource = new HikariDataSource(config);
```

Si utilizas un marco como Spring en Java, esto se configura y gestiona automáticamente en segundo plano por ti.

Transacciones

Una transacción de base de datos es una unidad lógica de trabajo que contiene una o más operaciones de base de datos (como insertar, actualizar, eliminar) ejecutadas como una única operación atómica que o bien se realiza correctamente o bien falla por completo. La transacción mantiene la integridad de los datos al garantizar que no se apliquen cambios parciales. El siguiente ejemplo muestra una transferencia de dinero entre cuentas, utilizando una transacción para garantizar que ambas operaciones se realicen correctamente o fallen, manteniendo la integridad del saldo de la cuenta:

```
// Concise transaction example
try (Connection conn = dataSource.getConnection()) {
    conn.setAutoCommit(false);

try (PreparedStatement withdrawStmt =
        conn.prepareStatement(
            "UPDATE accounts SET balance = balance - ? WHERE id = ?");
     PreparedStatement depositStmt =
        conn.prepareStatement(
            "UPDATE accounts SET balance = balance + ? WHERE id = ?")) {

    // Withdraw from source account
    withdrawStmt.setBigDecimal(1, amount);
    withdrawStmt.setLong(2, fromAccountId);
    withdrawStmt.executeUpdate();

    // Deposit to destination account
    depositStmt.setBigDecimal(1, amount);
    depositStmt.setLong(2, toAccountId);
    depositStmt.executeUpdate();

    conn.commit();
} catch (SQLException e) {
    conn.rollback();
    throw e;
}
```

NOTA

La responsabilidad de aplicar las transacciones depende de tu sistema de base de datos y de la configuración del cliente. Aunque las bases de datos suelen garantizar las propiedades ACID para las transacciones correctamente definidas (como veremos en la siguiente sección), el desarrollador de la aplicación suele ser responsable de definir los límites de las transacciones. Deberás asegurarte de que las operaciones relacionadas se agrupen dentro del mismo ámbito de transacción.

Modelos de consistencia y estrategias de almacenamiento en caché

Al principio de este capítulo aprendiste que los datos son la columna vertebral de la mayoría de las aplicaciones en las que trabajas. Esto significa que la base de cualquier sistema reside en su capacidad para gestionar los datos de forma fiable y ofrecer un alto rendimiento. En esta sección se exploran dos conceptos fundamentales que abordan estas necesidades: los modelos de consistencia y las estrategias de almacenamiento en caché. Aunque a menudo se tratan por separado, estos elementos funcionan conjuntamente en las aplicaciones modernas basadas en datos para crear un equilibrio entre la fiabilidad de los datos y la velocidad.

Modelos de consistencia

Basándose en los conceptos de transacción « » que aprendiste anteriormente, *los modelos de consistencia* definen cómo tu aplicación gestionará la precisión de los datos en las diferentes partes de tu sistema.

Mientras que las transacciones mantienen la integridad de los datos en operaciones individuales, los modelos de consistencia determinan cómo se mantiene esa integridad en varios servidores o cuando varios usuarios acceden a la misma información simultáneamente.

La consistencia en las bases de datos se refiere a la garantía de que cualquier transacción llevará la base de datos de un estado válido a otro, asegurando que todos los datos se adhieran a las reglas, restricciones y relaciones definidas sin contradicciones. Este principio es una de las cuatro propiedades ACID (atomicidad, consistencia, aislamiento y durabilidad) que garantizan un procesamiento fiable de las transacciones en los sistemas de bases de datos.

Tipos de modelos de consistencia

Las aplicaciones tienen diferentes necesidades en cuanto a la rapidez con la que los cambios en los datos deben aparecer en todo el sistema. Las bases de datos ofrecen varios modelos de consistencia para satisfacer estos requisitos:

Consistencia fuerte

Garantiza que cada parte de tu sistema vea los mismos datos al mismo tiempo, y que los cambios aparezcan en el orden exacto en que se produjeron. En PostgreSQL, cuando actualizas el saldo de la cuenta de un usuario, todas las lecturas posteriores reflejan inmediatamente ese cambio. Como resultado, el sistema puede bloquear temporalmente las operaciones para mantener esta garantía, dando prioridad a la precisión sobre la disponibilidad.

Consistencia eventual

Da prioridad al funcionamiento de la aplicación permitiendo inconsistencias temporales que se resuelven con el tiempo. En Amazon DynamoDB, la actualización del precio de un producto puede tardar unos segundos en propagarse a nivel global. El resultado es que los usuarios de diferentes regiones pueden ver precios diferentes durante un breve periodo de tiempo, pero el sistema sigue estando disponible durante las interrupciones del servicio. Este modo funciona bien cuando la consistencia inmediata no es crítica.

Consistencia causal

Mantiene el orden solo entre operaciones relacionadas causalmente. En herramientas de edición colaborativa como Google Docs, si el usuario A comenta la edición del usuario B, todos ven la edición antes del comentario, pero las ediciones no relacionadas del usuario C pueden aparecer en diferentes órdenes para diferentes usuarios. Esto proporciona un término medio entre la consistencia fuerte y la eventual.

Consistencia de sesión

Garantiza la consistencia dentro de las sesiones individuales de los usuarios, al tiempo que permite diferencias entre ellos. En las aplicaciones de comercio electrónico, una vez que añades un artículo a tu carrito, siempre lo verás allí durante tu sesión, incluso si las vistas del inventario de otros usuarios pueden estar ligeramente desactualizadas. Esto crea una experiencia consistente para cada usuario sin necesidad de sincronización global.

El teorema CAP y sus implicaciones

El *teorema CAP* explica una compensación fundamental en los sistemas distribuidos. Cuando tus datos se distribuyen entre varios servidores, solo puedes garantizar dos de estas tres propiedades simultáneamente:

Consistencia

Todos los servidores muestran los mismos datos al mismo tiempo.

Disponibilidad

El sistema sigue funcionando incluso si algunos servidores fallan.

Tolerancia a particiones

El sistema sigue funcionando incluso si se interrumpen las conexiones de red entre los servidores.

Estas compensaciones dan lugar a los siguientes diseños de sistemas:

Sistemas CP

Los sistemas CP dan prioridad a la coherencia y la tolerancia a particiones a expensas de la disponibilidad. Si hay un problema de red, el sistema puede dejar de aceptar solicitudes en lugar de arriesgarse a mostrar datos incorrectos. Algunos ejemplos son las aplicaciones bancarias tradicionales, los sistemas de gestión de inventario y los servicios financieros, donde la precisión de los datos es fundamental. Tecnologías como Apache HBase y Google Spanner entran en esta categoría.

Sistemas AP

Los sistemas AP se centran en la disponibilidad y la tolerancia a particiones, aceptando la consistencia eventual. Estos sistemas son ideales para redes de distribución de contenidos, feeds de redes sociales y motores de recomendación, donde la consistencia inmediata es menos crítica que el tiempo de actividad del sistema. Tecnologías como Amazon DynamoDB y Cassandra son buenos ejemplos de este enfoque. Es mejor mostrar una publicación en redes sociales ligeramente desactualizada que hacer que todo el feed no esté disponible.

Sistemas CA

Los sistemas CA ofrecen consistencia y disponibilidad, pero no toleran las particiones de red. Estos sistemas funcionan bien en bases de datos de un solo nodo o en clústeres estrechamente acoplados en redes fiables. Los sistemas de gestión de bases de datos relacionales (RDBMS) tradicionales, como MySQL o PostgreSQL, configurados sin capacidades distribuidas, funcionan en este modo.

Elegir el modelo de consistencia adecuado

Las aplicaciones modernas suelen emplear múltiples modelos de consistencia para diferentes tipos de datos dentro del mismo sistema. La clave aquí es hacer coincidir los requisitos de consistencia con el impacto en el negocio.

Utiliza una consistencia fuerte para operaciones en las que la precisión es fundamental y la indisponibilidad temporal es aceptable. Las transacciones financieras, las actualizaciones de inventario y la autenticación de usuarios suelen requerir este enfoque.

Utiliza la consistencia eventual para los datos en los que se aceptan ligeros retrasos y la disponibilidad es crucial. Los registros de actividad de los usuarios, los sistemas de recomendación y las fuentes de contenido pueden tolerar inconsistencias temporales a cambio de un mejor rendimiento y tiempo de actividad.

Utiliza la consistencia de sesión para las funciones orientadas al usuario en las que la consistencia individual es más importante que la sincronización global. Los carritos de la compra, las preferencias de usuario y los borradores de contenido funcionan bien con este modelo.

A la hora de elegir el modelo de consistencia adecuado, recuerda que no es permanente. La parte importante de este proceso es comprender los requisitos tal y como existen en este momento, equilibrar las ventajas y desventajas y seleccionar el modelo adecuado.

Optimización del rendimiento con el almacenamiento en caché

El almacenamiento en caché es una técnica que almacena los datos de uso frecuente en ubicaciones de almacenamiento más rápidas para reducir el tiempo de respuesta y la carga de la base de datos. Es como mantener las aplicaciones más utilizadas en la pantalla de inicio de tu teléfono en lugar de buscar entre todas tus aplicaciones cada vez.

Cuando tu aplicación solicita datos, la caché sirve como una capa de almacenamiento de acceso rápido entre tu aplicación y la base de datos.

Cuando realizas una llamada a la base de datos, incluso una sencilla, se producen llamadas de red y lecturas de disco que pueden tardar decenas o cientos de milisegundos. Una caché almacena datos en la memoria, lo que reduce los tiempos de respuesta entre 10 y 100 veces o más, dependiendo del contexto y la arquitectura.

Sin embargo, el almacenamiento en caché tiene sus inconvenientes. Introduce complejidad en cuanto a la actualidad de los datos y añade otro punto potencial de fallo. Por eso es fundamental definir una estrategia de almacenamiento en caché. Una buena estrategia determina cómo tu aplicación gestiona la relación entre los datos almacenados en caché y la fuente de verdad de tu base de datos.

Estrategias comunes de almacenamiento en caché

Las aplicaciones modernas utilizan tres estrategias principales de almacenamiento en caché, cada una con diferentes compensaciones entre rendimiento, consistencia y complejidad:

Caché aparte (carga diferida)

Con el almacenamiento en caché aparte ( ), el código de tu aplicación se encarga de gestionar tanto la caché como la base de datos. Cuando se realiza una solicitud de datos, tu aplicación comprueba primero la caché para ver si están disponibles. Si los datos no están disponibles (*fallo de caché*), la aplicación recupera los datos de la base de datos y los almacena en la caché para

futuras solicitudes. Si los datos están disponibles (*acierto de caché*), la aplicación los recupera de la caché.

En el siguiente ejemplo, se busca una entrada de blog en la caché por su ID. Si se encuentra el ID, la aplicación devolverá esa entrada. Si no es así, la aplicación buscará la entrada del blog en la base de datos y luego la almacenará en la caché para la próxima solicitud:

```
public BlogPost getPostById(Long id) {
    BlogPost cached = cache.get(id);
    if (cached != null) return cached;

    BlogPost post = database.findById(id);
    cache.put(id, post);
    return post;
}
```

Esta estrategia funciona bien para aplicaciones con gran volumen de lectura en las que se desea un control preciso sobre lo que se almacena en caché. La desventaja es que el código de la aplicación se vuelve más complejo y la primera solicitud de cualquier dato siempre será más lenta debido a la búsqueda y la falta de caché.

Escritura directa

Con el almacenamiento en caché de escritura directa, cada operación de escritura actualiza simultáneamente tanto la caché como la base de datos. Esto garantiza que tanto la base de datos

como la caché contengan los mismos datos, pero puede ralentizar el rendimiento, ya que la escritura se produce en ambas ubicaciones. En el siguiente ejemplo, la base de datos y la caché se actualizan dentro del mismo método:

```
public void updatePost(BlogPost post) {
    database.save(post);
    cache.put(post.getId(), post);
}
```

Esta estrategia funciona bien cuando necesitas una fuerte consistencia entre la caché y la base de datos, y cuando el rendimiento de lectura es más importante que el rendimiento de escritura. La desventaja es un rendimiento más lento en las escrituras y el posible almacenamiento en caché de datos que podrían no leerse con frecuencia.

Escritura diferida (escribir detrás)

El almacenamiento en caché con escritura diferida escribe los datos en la caché inmediatamente, pero actualiza la base de datos de forma asíncrona. Esto proporciona un rendimiento rápido en las operaciones de escritura, pero introduce el riesgo de pérdida de datos si la caché falla antes de que se actualice la base de datos. En el siguiente ejemplo, se actualiza la caché y, a continuación, se genera un proceso en segundo plano para actualizar la base de datos:

```
public void updatePost(BlogPost post) {
    cache.put(post.getId(), post);
    asyncQueue.schedule(() -> database.save(post));
}
```

> Esta estrategia funciona bien para aplicaciones con gran volumen de escritura que pueden asumir cierto riesgo de pérdida de datos a cambio de un mejor rendimiento. Algunos ejemplos de la estrategia de escritura diferida son la actividad de los usuarios, el registro o las métricas, en los que es aceptable perder algunos puntos de datos.

Cuándo utilizar el almacenamiento en caché

El almacenamiento en caché es una herramienta poderosa en tu arsenal, pero no está exento de compensaciones. Una caché es otra parte móvil de tu sistema que puede fallar, quedar obsoleta o consumir recursos. La cuestión no es si el almacenamiento en caché es bueno o malo, sino si los beneficios de añadirlo a tu aplicación justifican la complejidad añadida.

Piensa en el catálogo de productos de una tienda online. Cuando un usuario visita la tienda y busca un producto, la aplicación necesita recuperar información de varias tablas, como los detalles del producto, el inventario, las reseñas y los precios. Si cada página de producto tarda una media de 200 ms en completarse y 1000 compradores utilizan la tienda simultáneamente, tu aplicación podría realizar 4000 solicitudes a la base de datos en un breve periodo de tiempo. La base de datos

debe gestionar este gran volumen de consultas simultáneas y mantener unos tiempos de respuesta aceptables.

La mayoría de estas solicitudes son para obtener información que rara vez cambia. En este caso, el almacenamiento en caché tiene sentido porque puedes atender las solicitudes en un promedio de 5 ms desde la memoria, mientras que tu base de datos se encarga de las operaciones verdaderamente dinámicas, como el procesamiento de pedidos.

Comparemos esto con otro escenario, como una plataforma de negociación en tiempo real en la que los precios cambian varias veces por segundo. Incluso si establecieras una política de expulsión de la caché (eliminación de datos de la caché) a intervalos de 30 segundos, seguirías mostrando precios obsoletos, lo que podría suponer un coste para tus usuarios. La complejidad de la invalidación de la caché y el riesgo de mostrar datos obsoletos superan con creces las ventajas de rendimiento.

A la hora de evaluar si el almacenamiento en caché es adecuado para un escenario concreto, hazte las siguientes preguntas:

- ¿Con qué frecuencia cambian estos datos?
- ¿Cuáles son los costes de proporcionar datos obsoletos a los usuarios?
- ¿Cuánto cuestan tus consultas actuales?
- ¿Existen actualmente cuellos de botella en el rendimiento?

Las respuestas a estas preguntas te guiarán hacia la solución adecuada para tu caso de uso específico.

Almacenamiento en caché y coherencia

El almacenamiento en caché plantea sus propios retos en lo que respecta a la coherencia, como has aprendido anteriormente en esta sección. Cuando almacenas datos en caché, estás creando una copia temporal que puede quedar obsoleta cuando cambian los datos originales. Las diferentes estrategias de almacenamiento en caché gestionan esto de forma diferente:

- *El almacenamiento en caché con TTL (tiempo de vida)* proporciona consistencia eventual al caducar automáticamente los datos almacenados en caché después de un tiempo determinado.
- *El almacenamiento en caché con escritura directa* mantiene una fuerte consistencia entre la caché y la base de datos, pero a costa del rendimiento de la escritura.
- *El almacenamiento en caché de escritura diferida* acepta inconsistencias temporales en favor del rendimiento.

La clave está en sopesar las ventajas y desventajas con respecto a los requisitos de tu negocio. En el mundo financiero, los datos pueden requerir un almacenamiento en caché con escritura directa para garantizar una consistencia sólida, mientras que un blog personal podría utilizar el almacenamiento en caché con consistencia eventual a través de la expiración del TTL.

CONSIDERACIONES SOBRE LOS SISTEMAS DISTRIBUIDOS

Cuando tu aplicación se escala a través de varios servidores, te enfrentas a retos adicionales:

- La consistencia de la caché se vuelve más compleja.
- Puede ser necesario realizar transacciones distribuidas.
- Es posible que tengas que implementar patrones de consistencia eventual.
- Las estrategias de replicación y fragmentación de datos cobran importancia.

Estos temas merecen un capítulo propio, pero ten en cuenta que, a medida que tu aplicación crezca, tu estrategia de persistencia tendrá que evolucionar.

Planificación para el crecimiento de los datos

Como has aprendido a lo largo de este capítulo, no existe un plan «perfecto» en lo que respecta a tus datos, pero debes evitar actuar sin ningún plan. Dedicar algo de tiempo a pensar en tu estrategia de datos desde el principio puede ahorrarte mucho esfuerzo, tiempo y estrés más adelante.

Planificar el crecimiento significa revisar periódicamente las métricas de rendimiento y las necesidades de capacidad. Lo mejor es hacerlo de forma conjunta entre el equipo de desarrollo, los administradores de bases de datos y el personal de operaciones. Presta atención a indica-

dores clave como los tiempos de respuesta de las consultas que superan los umbrales aceptables, la CPU de la base de datos constantemente por encima de un determinado porcentaje, el uso de la memoria en constante aumento o el incremento de las quejas de los usuarios sobre la lentitud de los tiempos de carga. Estas conversaciones deben tener lugar mensualmente para las aplicaciones en crecimiento, lo que te permitirá abordar los problemas antes de que se conviertan en críticos.

Uso de estrategias de escalado

Cuando aparecen indicadores de crecimiento, tienes varias opciones, cada una de ellas adecuada para diferentes situaciones:

Escalado vertical

Añade más recursos (CPU, RAM) a tu servidor de base de datos. A menudo, esta es la primera medida que se toma, ya que es sencilla y no requiere cambios arquitectónicos, pero con el tiempo se alcanzarán los límites del hardware.

Escalado horizontal

Distribuye tus datos entre varios servidores. Esto se hace necesario cuando el escalado vertical alcanza sus límites o cuando necesitas una distribución geográfica para aplicaciones globales.

Réplicas de lectura

Crea copias de tu base de datos para operaciones de lectura, lo que reduce la carga en la base de datos principal. Considera este enfoque primero para aplicaciones con gran volumen de

lectura. Es más fácil de implementar que el escalado horizontal completo y puede proporcionar mejoras significativas en el rendimiento.

Fragmentación

Divide tus datos en varias bases de datos basándose en una clave de fragmentación. Esta es la opción más compleja, pero necesaria para aplicaciones con grandes volúmenes de datos que no caben en un solo servidor.

RÉPLICAS DE LECTURA FRENTE A ALMACENAMIENTO EN CACHÉ

Aunque las réplicas de lectura y el almacenamiento en caché mejoran el rendimiento de lectura, funcionan de manera diferente:

- *Las réplicas de lectura* son copias completas de tu base de datos que se mantienen sincronizadas con la base de datos principal mediante la replicación.
- *El almacenamiento en caché* guarda los datos a los que se accede con frecuencia en la memoria como una copia temporal que puede quedar obsoleta.

Las réplicas de lectura proporcionan consistencia eventual a nivel de la base de datos y no requieren cambios en el código de la aplicación, mientras que el almacenamiento en caché requiere que tu aplicación gestione la invalidación y la consistencia de la caché. Las réplicas de lectura son especialmente útiles para la distribución geográfica y las cargas de trabajo con gran volumen de lectura, mientras que el almacenamiento en caché destaca por reducir los tiempos de respuesta de los datos a los que se accede con frecuencia.

Mantener el rendimiento durante el crecimiento

El mantenimiento regular de los datos cobra cada vez más importancia a medida que tu aplicación se escala. Puedes mantener el rendimiento de la siguiente manera:

- Implementa estrategias de archivado de datos antiguos para mantener los conjuntos de trabajo manejables.
- Configura índices de base de datos adecuados y revísalos periódicamente a medida que evolucionen los patrones de consulta.
- Programa tareas periódicas de mantenimiento de la base de datos, como analizar el rendimiento de las consultas y optimizar las consultas lentas.

Comprender los distintos tipos de bases de datos y los niveles de abstracciones de persistencia puede ayudarte a tomar decisiones informadas sobre la arquitectura y la dirección de tu aplicación. La elección correcta depende de los requisitos de tu aplicación y de las habilidades de tu equipo, y no existe una solución universal que sea la mejor.

Consultas y gestión del rendimiento de los datos

Una aplicación lenta frustrará a los usuarios, por muy buena que sea su apariencia. Aunque el equipo de backend se centra en proporcionar datos a los usuarios, a veces se olvida del rendimiento; el equipo está satisfecho siempre que la aplicación funcione. Tanto si estás creando una pequeña aplicación web como un complejo sistema empresarial, la forma en que consultas y gestionas los datos puede determinar el rendimiento de tus aplicaciones.

En esta sección se tratan los fundamentos del acceso eficiente a los datos, que pueden ayudarte a evitar problemas de rendimiento en el fu-

turo. Estos principios básicos son tan comunes que los encontrarás en muchas aplicaciones. Aprenderás técnicas prácticas para optimizar las consultas, comprenderás la indexación de bases de datos y descubrirás cuándo utilizar herramientas de rendimiento específicas. Estas habilidades no solo mejorarán el rendimiento de tus aplicaciones, sino que también demostrarán tu profundo conocimiento de los fundamentos de las bases de datos.

Redacción eficiente de consultas

Cuando empiezas, solo quieres escribir la consulta que te dé los resultados que buscas. El siguiente paso debería ser preguntarte: «¿Cómo puedo mejorar el rendimiento de esta consulta?». Comprender cómo escribir consultas eficientes es una habilidad fundamental que repercute enormemente en el rendimiento de tu aplicación. Una consulta mal optimizada puede consumir recursos excesivos del servidor, crear cuellos de botella y provocar una experiencia de usuario frustrante.

La diferencia entre una consulta bien optimizada y una mal optimizada puede ser enorme, ya que a veces reduce el tiempo de ejecución de segundos a milisegundos. Exploremos algunas técnicas esenciales que te ayudarán a optimizar tu acceso a los datos y a crear aplicaciones que sigan respondiendo incluso con un alto rendimiento.

Optimización básica de consultas

El primer paso para escribir consultas eficientes es comprender qué las hace ineficientes en primer lugar. Considera este escenario común:

```
public List<User> getAllActiveUsers() {
    return jdbcTemplate.query(
        "SELECT * FROM users WHERE active = true",
        (rs, rowNum) -> {
            User user = new User();
            user.setId(rs.getLong("id"));
            user.setUsername(rs.getString("username"));
            user.setEmail(rs.getString("email"));
            user.setActive(rs.getBoolean("active"));
            user.setCreatedAt(rs.getTimestamp("created_at"));
            // ... mapping 15 more fields
            return user;
        });
}
```

Esta consulta parece bastante inocente, pero tiene dos problemas importantes. En primer lugar, `SELECT *` recupera todas las columnas de la base de datos, incluso si solo necesitas una o dos. En segundo lugar, está cargando potencialmente miles de registros en la memoria a la vez. Mejorémosla:

```
public List<UserSummary> getActiveUserSummaries() {
    return jdbcTemplate.query(
        "SELECT id, username, email FROM users WHERE active = true LIMIT 100",
        (rs, rowNum) -> {
            UserSummary summary = new UserSummary();
            summary.setId(rs.getLong("id"));
            summary.setUsername(rs.getString("username"));
            summary.setEmail(rs.getString("email"));
            return summary;
```

```
        });
}
```

Al seleccionar solo las columnas que necesitas y limitar el conjunto de resultados, has reducido drásticamente la cantidad de datos transferidos desde la base de datos a tu aplicación. Este simple cambio puede mejorar el rendimiento en varios órdenes de magnitud, especialmente cuando se trata de tablas grandes.

CONSEJO

Sé siempre específico con respecto a las columnas que seleccionas y considera la posibilidad de utilizar *la paginación* (dividir los resultados en fragmentos más pequeños, como 20 registros por página) cuando recuperes grandes conjuntos de datos. Esto no solo mejora el rendimiento, sino que también facilita el mantenimiento de tu código al documentar claramente qué datos necesita exactamente tu aplicación.

Sentencia preparada

Una vez que hayas realizado algunas optimizaciones básicas de las consultas, el siguiente paso es asegurarte de que la base de datos pueda ejecutarlas de la forma más eficiente posible. Una *sentencia preparada* es una consulta SQL precompilada que te permite insertar valores de datos de forma segura en tiempo de ejecución, al tiempo que mejora el rendimiento y evita la inyección de SQL. En el siguiente ejemplo, la sentencia se construye concatenando valores directamente en SQL:

```
public User findUserByEmailAndStatus(String email, boolean active) {
    String sql = "SELECT id, username, email FROM users WHERE email = '" +
                 email + "' AND active = " + active;
    return jdbcTemplate.queryForObject(sql, User.class);
}
```

Cada vez que se ejecuta este método, la base de datos debe analizar, compilar y optimizar una consulta completamente nueva. Ahora compáralo con una instrucción preparada:

```
public User findUserByEmailAndStatus(String email, boolean active) {
    return jdbcTemplate.queryForObject(
        "SELECT id, username, email FROM users WHERE email = ? AND active = ?",
        new Object[]{email, active},
        User.class);
}
```

Con las sentencias preparadas, la base de datos analiza y optimiza la consulta una vez, y luego reutiliza ese plan de ejecución para llamadas posteriores con diferentes parámetros. Este almacenamiento en caché del plan de consulta puede reducir el tiempo de ejecución entre un 20 % y un 50 % para las consultas que se ejecutan con frecuencia.

Más allá del rendimiento, las sentencias preparadas gestionan automáticamente el escape de parámetros, protegiendo tu aplicación de ataques de inyección SQL sin necesidad de sanitizar manualmente las entradas.

Gestión de índices

Imagina que intentas encontrar menciones relevantes de «almacenamiento en caché» en todo este libro sin un índice. Tendrías que leer cada página y tomar notas a medida que avanzas. Los índices de bases de datos resuelven el mismo problema creando una tabla de búsqueda que asigna los valores directamente al lugar donde se almacenan, transformando los lentos escaneos de tablas completas en recuperaciones rápidas. En el siguiente ejemplo, la consulta no tiene índices y tardará aproximadamente 2000 ms en ejecutarse:

```
// Query execution time without an index: ~2000ms
UserProfile profile = jdbcTemplate.queryForObject(
    "SELECT * FROM user_profiles WHERE email = ?",
    new Object[]{"john.doe@example.com"},
    UserProfile.class);
```

Sin un índice en la columna de correo electrónico, la base de datos realiza un escaneo completo de la tabla. Ahora, agreguemos un índice:

```
CREATE INDEX idx_user_profiles_email ON user_profiles(email);
```

Con este índice, la misma consulta podría ejecutarse ahora en 5 ms en lugar de 2000 ms, ¡una mejora de 400 veces! Sin embargo, los índices no son gratuitos. Ocupan espacio de almacenamiento y ralentizan las operaciones de escritura porque la base de datos debe actualizar cada índice cuando cambian los datos.

A la hora de decidir qué indexar, ten en cuenta estas directrices:

- Indexa las columnas que se utilizan con frecuencia en las cláusulas « `WHERE` ».
- Indexa las columnas utilizadas en condiciones « `JOIN` ».
- Indexa las columnas utilizadas en las cláusulas « `ORDER BY` » o « `GROUP BY` ».
- Considera la posibilidad de utilizar índices compuestos para las consultas que filtran por varias columnas.
- Evita indexar columnas con baja cardinalidad (pocos valores únicos).

Manejo de conjuntos de resultados grandes

Cuando trabajas en desarrollo, uno de los problemas más fáciles de pasar por alto es el gran volumen de datos que puede existir en tu entorno de producción. Sin un conjunto de datos grande, es fácil olvidar que devolver grandes colecciones de objetos en producción puede causar problemas de rendimiento. Como se mencionó anteriormente, una forma de abordar esto es limitando el número de registros devueltos en una consulta. Aquí es donde entra en juego la paginación, ya que te permite navegar por los resultados en fragmentos manejables.

En el siguiente ejemplo, un método Java recupera datos de productos paginados filtrados por categoría. El método ejecuta dos consultas a la base de datos: una para obtener la página específica de productos y otra para obtener el recuento total. A continuación, el método reúne estos resultados en un objeto de página que contiene tanto los datos

del producto como los metadatos de paginación necesarios para la aplicación cliente:

```
public Page<Product> getProductsByCategory(String category, int page, int size) {
    int offset = page * size;

    List<Product> products = jdbcTemplate.query(
        "SELECT id, name, price FROM products WHERE category = ? " +
        "ORDER BY name LIMIT ? OFFSET ?",
        new Object[]{category, size, offset},
        (rs, rowNum) -> {
            Product product = new Product();
            product.setId(rs.getLong("id"));
            product.setName(rs.getString("name"));
            product.setPrice(rs.getBigDecimal("price"));
            return product;
        });

    int totalCount = jdbcTemplate.queryForObject(
        "SELECT COUNT(*) FROM products WHERE category = ?",
        new Object[]{category},
        Integer.class);

    return new Page<>(products, page, size, totalCount);
}
```

Este enfoque tiene un posible problema de rendimiento: la segunda consulta que cuenta todas las filas coincidentes puede resultar costosa a medida que la tabla crece. El código utiliza una paginación basada en desplazamientos, lo que requiere que la base de datos escanee y des-

carte todas las filas anteriores al punto de desplazamiento, lo que se vuelve cada vez más ineficiente con desplazamientos más grandes.

Aquí tienes algunas alternativas a tener en cuenta:

- Reemplaza la paginación por desplazamiento con la paginación por conjunto de claves (utilizando `WHERE id > last_seen_id`).
- Almacena en caché los resultados del recuento cuando no cambien con frecuencia.
- Estima los recuentos para conjuntos de datos muy grandes.
- Ten en cuenta patrones de interfaz de usuario como el desplazamiento infinito, que no requieren recuentos totales.

Herramientas y buenas prácticas

Piensa en el viejo adagio: «Si un árbol cae en un bosque y no hay nadie cerca para oírlo, ¿hace ruido?». Ahora imagina que tus usuarios están en ese bosque y oyen caer el árbol en tu base de datos mientras tú estás en casa, sin saber nada de lo que está pasando. Inevitablemente surgirán problemas, pero la pregunta clave es: ¿cómo vas a obtener visibilidad de esos problemas?

Comprender los planes de ejecución de consultas

Un planificador de consultas de bases de datos (también llamado *optimizador*) es un componente crítico de cualquier sistema de gestión de bases de datos que traduce tus sentencias SQL en programas ejecutables llamados *planes de ejecución*. Piensa en él como un compilador

que toma tu código SQL y determina la forma más eficiente de recuperar los datos solicitados.

¿Qué es un planificador de consultas de bases de datos?

Los planificadores de consultas funcionan analizando tu consulta SQL, teniendo en cuenta los índices disponibles, las estadísticas de las tablas y diversas estrategias de ejecución para generar un plan óptimo. Hay dos tipos principales de planificadores de consultas: los optimizadores basados en reglas ( ) siguen reglas estrictas (como utilizar siempre los índices disponibles) y los optimizadores basados en costes (cost-based optimizers) generan múltiples planes de ejecución y seleccionan el que tiene el coste estimado más bajo. La mayoría de los sistemas de bases de datos modernos utilizan optimizadores basados en costes.

Planificadores de consultas específicos de bases de datos

Cada sistema de base de datos tiene su propio planificador de consultas con características únicas:

PostgreSQL

El planificador/optimizador de PostgreSQL crea un plan de ejecución generando posibles planes para escanear cada relación (tabla) en la consulta, determinando qué índices utilizar y examinando diferentes secuencias de unión para encontrar la más económica. Para consultas complejas con muchas uniones, PostgreSQL utiliza un optimizador de consultas genético para encontrar un plan razonable (aunque no necesariamente óptimo) en un tiempo razonable.

MySQL

El optimizador de consultas de MySQL también utiliza un enfoque basado en el coste, centrándose en optimizar las operaciones de lectura, donde destaca especialmente. Evalúa diferentes métodos de acceso, tipos de unión y órdenes de unión para encontrar el plan de ejecución más eficiente.

Oracle

El optimizador de Oracle es muy sofisticado y ofrece características adicionales de nivel empresarial para la optimización, incluidos planes de ejecución adaptativos que pueden cambiar durante la ejecución de la consulta en función de las estadísticas de tiempo de ejecución.

Uso de planes de ejecución de consultas

La mayoría de las bases de datos proporcionan el comando `EXPLAIN` para ver el plan de ejecución de una consulta. A continuación se explica cómo utilizarlo:

```
 -- Basic EXPLAIN in PostgreSQL
EXPLAIN SELECT u.username, p.bio
FROM users u JOIN profiles p ON u.id = p.user_id
WHERE u.active = true;

-- More detailed analysis with EXPLAIN ANALYZE (PostgreSQL)
EXPLAIN ANALYZE SELECT u.username, p.bio
FROM users u JOIN profiles p ON u.id = p.user_id
WHERE u.active = true;
```

```
-- MySQL EXPLAIN format
EXPLAIN SELECT u.username, p.bio
FROM users u JOIN profiles p ON u.id = p.user_id
WHERE u.active = true;
```

`EXPLAIN` Es compatible con PostgreSQL, MySQL, Oracle y otros sistemas de bases de datos importantes, aunque la sintaxis y el formato de salida pueden variar. PostgreSQL y MySQL también ofrecen `EXPLAIN ANALYZE` , que ejecuta la consulta y proporciona información adicional sobre cómo las expectativas del optimizador coincidieron con la ejecución.

La salida del plan de ejecución suele mostrar lo siguiente:

- ¿A qué tablas se accederá y en qué orden?
- Qué métodos de unión se utilizarán (bucle anidado, unión por fusión, unión por hash)
- ¿Qué índices (si los hay) se utilizarán para cada tabla?
- Qué tipos de exploraciones se realizarán (exploración secuencial, exploración de índices)
- Costes estimados y/o reales en términos de tiempo y recursos
- Recuento de filas (estimado frente a real cuando se utiliza `ANALYZE`)

Aprender a leer los planes de ejecución requiere práctica, pero es una de las habilidades más valiosas para depurar consultas lentas. Al analizar los planes, busca señales de advertencia como estas:

- Escaneos completos de tablas cuando se deben utilizar índices

- Métodos de unión ineficaces para el volumen de datos
- Índices que faltan o no se utilizan
- Costes estimados o recuentos de filas elevados
- Grandes discrepancias entre los valores estimados y los reales (cuando se utiliza `ANALYZE`)

El uso regular de `EXPLAIN` te ayuda a comprender cómo piensa y toma decisiones tu base de datos, lo que te permite escribir consultas más eficientes y diseñar mejores esquemas e índices.

NOTA

Más allá de la optimización de consultas individuales, la comparación de planes de ejecución entre entornos (desarrollo, staging, producción) puede revelar problemas críticos de implementación. Las inconsistencias del entorno, como índices que faltan, estadísticas obsoletas o distribuciones de datos diferentes, pueden hacer que las consultas que funcionan bien en las pruebas fallen en producción. Cuando el rendimiento se degrada repentinamente después de una implementación, la comparación de planes de ejecución entre entornos a menudo revela la causa raíz más rápidamente que otros enfoques de depuración.

Monitoreo y análisis de bases de datos

Es importante comprender cómo se comportan tus consultas en condiciones ideales utilizando `EXPLAIN`, pero el rendimiento en el mundo real depende de cómo interactúa tu aplicación con la base de datos bajo carga. Confiar únicamente en herramientas nativas de la base de

datos, como los registros de consultas lentas o las tablas de estadísticas (`pg_stat_statements`), solo ofrece una visión parcial.

Para obtener una visión holística, es necesario incorporar el monitoreo desde la perspectiva de la aplicación. Aquí es donde entra en juego la observabilidad a nivel de aplicación, que proporciona información sobre cómo las interacciones con la base de datos afectan al estado y al rendimiento general de la aplicación. En ingeniería de software, la observabilidad se refiere a la capacidad de comprender el estado interno de un sistema basándose en los resultados que produce.

Los marcos de aplicaciones modernos suelen proporcionar potentes herramientas para la observabilidad, integrando métricas, registros y rastreo. Spring Boot, a través de su módulo Actuator y la biblioteca de métricas Micrometer, destaca en esto, ya que ofrece información detallada sobre las interacciones con la base de datos con una configuración mínima.

Estos son los conceptos clave de Spring Boot que debes conocer cuando se habla de observabilidad:

Registro

: Registro de eventos y errores, que pueden correlacionarse con métricas y trazas para diagnosticar problemas.

Métricas

: Recopilar datos cuantitativos sobre el rendimiento del sistema. En el caso de las bases de datos, esto incluye el uso del grupo de conexiones (conexiones activas, inactivas y pendientes), los

tiempos de ejecución de las consultas, los tiempos de transacción y las tasas de error.

Seguimiento

Seguimiento de una solicitud a medida que se propaga por diferentes partes de la aplicación e incluso por sistemas distribuidos. Esto ayuda a identificar dónde se produce la latencia, incluido el tiempo dedicado a las llamadas a la base de datos.

Estos son algunos escenarios reales en los que la observabilidad proporciona información útil:

¿Es el grupo de conexiones un cuello de botella?

Los valores altos de las métricas « `jdbc.connections.pending` » (Conexiones activas) indican que los subprocesos esperan demasiado tiempo para obtener una conexión. Si ves que « `jdbc.connections.active` » (Conexiones activas) alcanza constantemente el límite de « `jdbc.connections.max` » (Conexiones activas máximas), esto sugiere que el grupo es demasiado pequeño o que las conexiones no se liberan correctamente (posibles fugas o transacciones de larga duración).

¿Estamos desperdiciando recursos de la base de datos?

Si `jdbc.connections.active` es constantemente bajo, mientras que `jdbc.connections.idle` es alto, es posible que tu grupo sea demasiado grande y consuma memoria y recursos de la base de datos innecesarios.

¿Por qué es lenta una solicitud de usuario específica?

El rastreo distribuido puede mostrarte todo el ciclo de vida de una solicitud. Si un rastreo revela que el 90 % del tiempo de solicitud se dedica a un único intervalo de base de datos (que representa una consulta o transacción), sabrás exactamente dónde centrar tus esfuerzos de optimización (probablemente necesitarás `EXPLAIN` en esa consulta específica).

¿Existe un problema de consulta N + 1?

Un rastreo puede mostrar docenas de consultas rápidas y pequeñas a la base de datos que se ejecutan secuencialmente dentro de una operación lógica. Este clásico problema N + 1, que a menudo se origina en configuraciones erróneas del mapeo ORM, se hace inmediatamente visible en un rastreo.

¿Hay consultas específicas que se vuelven más lentas con el tiempo?

Al monitorear las métricas del tiempo de ejecución de las consultas (si está habilitado) y correlacionarlas con las implementaciones de aplicaciones o el crecimiento de los datos, puedes identificar de forma proactiva las consultas que necesitan optimización antes de que provoquen incidentes graves.

¿Cómo afectan los errores de la base de datos a los usuarios?

Correlacionar los registros de errores de la base de datos o los picos de métricas con las tasas de error a nivel de aplicación o los rastreos de solicitudes fallidas ayuda a comprender el impacto que tienen los problemas de la base de datos en los usuarios (por ejemplo, violaciones de restricciones, interbloqueos, problemas de conectividad).

`DataSource` Spring Boot Actuator expone puntos finales listos para la producción para el monitoreo, mientras que Micrometer proporciona una fachada de métricas de aplicaciones independiente del proveedor. Cuando se utilizan juntos, pueden instrumentar automáticamente tus beans de base de datos (como HikariCP, el predeterminado en Spring Boot) para recopilar muchas de las estadísticas vitales que hemos mencionado, proporcionando los datos sin procesar necesarios para obtener esta información.

Equilibrio entre complejidad y rendimiento

Como ingeniero de software, una de las habilidades más importantes que desarrollarás es la de tomar decisiones informadas sobre las compensaciones en tu código. Esto es especialmente evidente cuando se trabaja con bases de datos, donde a menudo es necesario equilibrar la legibilidad y la mantenibilidad del código con el rendimiento de las consultas. Exploremos este equilibrio con un escenario real relacionado con el procesamiento de pedidos.

En el siguiente ejemplo, veremos dos enfoques para recuperar datos de pedidos. El primer enfoque utiliza Spring Data JPA con encadenamiento de métodos y operaciones de flujo; es limpio y fácil de entender, pero puede que no funcione bien a gran escala. El segundo enfoque utiliza SQL sin procesar; es más eficiente, pero requiere un mantenimiento más cuidadoso:

```
// Simple but potentially inefficient approach
List<Order> recentOrders = /n
orderRepository.findByUserIdAndStatusOrderByCreatedAtDesc(
```

```
        userId, OrderStatus.COMPLETED);

    List<OrderDto> result = recentOrders.stream()
        .filter(order -> order.getTotal().compareTo(new BigDecimal("100.00")) > 0)
        .map(orderMapper::toDto)
        .collect(Collectors.toList());

    // More efficient but complex approach
    List<OrderDto> efficientResult = jdbcTemplate.query(
        """
        SELECT o.id, o.created_at, o.total, u.name AS user_name
        FROM orders o JOIN users u ON o.user_id = u.id
        WHERE o.user_id = ? AND o.status = ? AND o.total > 100.00
        ORDER BY o.created_at DESC
        """,
        new Object[]{userId, OrderStatus.COMPLETED.name()},
        (rs, rowNum) -> {
            OrderDto dto = new OrderDto();
            dto.setId(rs.getLong("id"));
            dto.setCreatedAt(rs.getTimestamp("created_at"));
            dto.setTotal(rs.getBigDecimal("total"));
            dto.setUserName(rs.getString("user_name"));
            return dto;
        });
```

El segundo enfoque es más eficiente porque traslada el filtrado a la base de datos y recupera solo los datos necesarios. Sin embargo, también es más complejo y está más estrechamente vinculado a la estructura de la base de datos.

A la hora de decidir entre ambos enfoques, ten en cuenta estos factores:

- La escala de tus datos
- Requisitos de rendimiento
- Gastos generales de mantenimiento
- Familiaridad del equipo con SQL
- Necesidades futuras de flexibilidad

A menudo, lo mejor es un enfoque híbrido: utilizar ORM para la mayoría de las operaciones y recurrir a SQL sin procesar para las rutas críticas para el rendimiento.

El acceso eficiente a los datos no solo consiste en realizar consultas rápidas, sino en comprender todo el ciclo de vida de los datos en tu aplicación. Al seleccionar solo los datos que necesitas, aprovechar los índices adecuados, utilizar la paginación para conjuntos de resultados grandes y monitorear el rendimiento, puedes crear aplicaciones que sigan respondiendo incluso a medida que crece tu e o de datos.

Migración y transformación de datos

La migración de datos es un reto común al que se enfrentan todos los ingenieros de software en algún momento. Los equipos de desarrollo necesitan actualizar regularmente los sistemas de bases de datos entre tipos de bases de datos (como migrar de bases de datos relacionales a NoSQL o viceversa), conectarse con servicios externos o combinar datos de múltiples fuentes. Estas tareas se producen con frecuencia a lo

largo de tu carrera. Comprender cómo manejar adecuadamente la migración y la transformación de datos es una habilidad fundamental.

En esta sección, aprenderás los aspectos esenciales de la migración y la transformación de datos, incluidas las estrategias para mover datos entre sistemas, gestionar los cambios de esquema y garantizar la integridad de los datos durante todo el proceso. Estas habilidades te ayudarán a afrontar estos retos con confianza y a evitar errores comunes que podrían provocar la pérdida o el deterioro de los datos.

Comprender los fundamentos del movimiento de datos

Como ingeniero de software, una de tus herramientas principales es tu capacidad para resolver problemas. A primera vista, el problema de mover datos de un sistema a otro puede parecer bastante sencillo. Todo lo que hay que hacer es copiar los datos de un sistema y pegarlos en otro, ¿verdad? Por desgracia, el proceso rara vez es tan sencillo. Una migración de datos satisfactoria requiere una planificación, ejecución y validación cuidadosas.

La buena noticia es que puedes utilizar algunas estrategias de migración realmente útiles como plantillas para trasladar tus datos de un sistema a otro. Al igual que todas las decisiones que tomas, la estrategia que elijas dependerá de factores como el volumen de datos, la complejidad del sistema, los periodos de inactividad disponibles, la tolerancia al riesgo y los requisitos empresariales. Comprender estos factores te ayudará a seleccionar el enfoque más adecuado para tu escenario de migración específico.

Migración big bang frente a migración por fases

La migración big bang traslada todos los datos de una sola vez, normalmente durante un periodo de inactividad. Este enfoque puede ser más sencillo, pero también conlleva muchos riesgos. Utilicemos el ejemplo del traslado de empleados de una base de datos a otra. En un enfoque big bang, se podrían trasladar todos los empleados de toda la base de datos a una nueva.

Por el contrario, una *migración por fases* traslada los datos por etapas. Probablemente sigas haciéndolo durante un periodo de inactividad, pero solo trasladas una parte de los datos, los validas y luego pasas a otra parte. En nuestro ejemplo de los empleados, tal vez solo se trasladen los empleados de RR. HH. en esta fase. Aquí es donde entra en juego una planificación cuidadosa, ya que ayudará a determinar cómo se dividirá esta migración por fases. Puedes empezar por el grupo o grupos que puedan sentir menos el impacto si algo sale mal.

Si nunca has trabajado en un proyecto de migración de datos, empezar con un enfoque por fases puede ser la estrategia más segura. Aunque puede llevar más tiempo, reduce significativamente el riesgo y te da la oportunidad de aprender desde el principio antes de pasar a datos más críticos.

NOTA

Si procesas estos conjuntos de datos en código, asegúrate de hacerlo por lotes para evitar sobrecargar la memoria del sistema. Un lote de entre 1000 y 10 000 registros suele ser un buen punto de partida, pero es aquí donde debes realizar pruebas de prueba de concepto (PoC) para determinar las capacidades de tu sistema.

Procesos ETL

ETL (extraer, transformar, cargar) es un proceso para mover datos entre sistemas:

Extraer

: Extrae datos del sistema de origen

Transformar

: Convierte los datos para que se ajusten al formato y los requisitos del sistema de destino

Cargar

: Inserta los datos transformados en el sistema de destino

Este enfoque de tres pasos define claramente los límites de cada proceso. Aunque se trate de pasos aislados como parte de un proceso más amplio, cada uno de ellos presenta sus propios retos. Durante la extracción, es posible que te encuentres con límites de velocidad o problemas de rendimiento en los sistemas de producción. La transforma-

ción suele presentar problemas de calidad de los datos que no eran visibles durante las pruebas de PoC más pequeñas. La carga puede provocar restricciones o fallos de validación en el sistema de destino.

Al implementar un proceso ETL, tendrás que invertir tiempo en un tratamiento detallado de los errores y en el registro de los mismos para detectar e identificar los problemas. Te interesará saber cuándo y por qué los registros no se han migrado, para poder iterar rápidamente el proceso.

Sincronización de datos

En algunos casos, tendrás que mantener varios sistemas sincronizados durante un periodo de transición. Volvamos al ejemplo de la base de datos de empleados en el que se adoptó un enfoque de migración por fases. Aunque esto mitiga el riesgo, plantea otro reto: a medida que se incorporan nuevos empleados, estos tendrán que existir en ambas bases de datos hasta que se complete la migración por fases.

Para los escenarios de sincronización, considera la posibilidad de utilizar lo siguiente:

- Colas de mensajes para gestionar las actualizaciones de forma asíncrona
- Captura de datos modificados (CDC) para realizar un seguimiento de las modificaciones de la base de datos
- Procesos de conciliación para identificar y corregir inconsistencias

En esta sección, has aprendido algunas estrategias de migración y transformación de datos para mover datos entre sistemas. En la siguiente sección, aprenderás a realizar cambios en el esquema de la base de datos subyacente.

Gestión de los cambios en el esquema

A medida que los proyectos evolucionan con el tiempo, los cambios en el esquema de la base de datos subyacente son inevitables. Las estructuras de las tablas pueden cambiar y los campos pueden añadirse, eliminarse, renombrarse o modificar su tipo. La capacidad de gestionar estos cambios preservando la integridad de los datos es importante para mantener la fiabilidad de las aplicaciones. En esta sección, aprenderás sobre el control de versiones para esquemas de bases de datos, herramientas de migración como Flyway, técnicas para la transformación de datos durante los cambios de esquema y buenas prácticas para realizar actualizaciones fluidas con una interrupción mínima del sistema.

Uso del control de versiones para estructuras de datos

Cierra los ojos e imagina todo tu código base almacenado sin control de versiones. Esa idea debería darte escalofríos. Del mismo modo que no desarrollarías software sin realizar un seguimiento de los cambios o sin habilitar una colaboración optimizada, tu esquema de base de datos merece la misma protección.

Sin control de versiones, corres el riesgo de perder el historial crítico, sobrescribir el trabajo de tus colegas y carecer de la capacidad de re-

vertir los cambios problemáticos. Tu esquema, la base de tu infraestructura de datos, requiere el mismo control de versiones cuidadoso que protege el código de tu aplicación.

Herramientas como Flyway, Liquibase y Rails Migrations proporcionan marcos para el control de versiones de los esquemas de bases de datos. Estas herramientas realizan un seguimiento de los cambios de esquema que se han aplicado a cada entorno, lo que garantiza la coherencia entre el desarrollo, las pruebas y la producción.

Veamos un ejemplo práctico con Flyway. Flyway adopta un enfoque sencillo para las migraciones de bases de datos. Básicamente, Flyway hace lo siguiente:

- Mantiene una tabla especial en tu base de datos (normalmente llamada `flyway_schema_history`) que realiza un seguimiento de las migraciones que se han aplicado
- Escanea una carpeta designada en busca de scripts de migración que sigan una convención de nomenclatura específica
- Compara los scripts disponibles con lo que está registrado en la tabla de historial
- Ejecuta cualquier script nuevo que aún no se haya aplicado

Por ejemplo, puedes organizar tus migraciones como archivos SQL con números de versión:

```
V1__Create_users_table.sql
V2__Add_email_to_users.sql
V3__Create_orders_table.sql
```

NOTA

La mayoría de las herramientas de migración, incluida Flyway, están diseñadas para ejecutar migraciones de forma secuencial. Esto significa que migrar de la versión 100 a la versión 250 requiere ejecutar cada script de migración en orden (100 → 101 → 102... → 250). Aunque esto pueda parecer ineficiente, en realidad es una característica de seguridad que garantiza la integridad de los datos y permite que cada migración se construya correctamente sobre el estado anterior. Algunos escenarios avanzados permiten «aplastar» las migraciones o crear rutas de migración directas, pero el enfoque secuencial es la práctica estándar porque garantiza la coherencia y facilita la resolución de problemas cuando surgen.

Las siguientes son las prácticas más importantes para el control de versiones de esquemas:

- Haz que cada script de cambio sea idempotente (se puede ejecutar varias veces sin causar daños).
- Asegura que los scripts sean compatibles con versiones anteriores siempre que sea posible.
- Incluye tanto el cambio como las transformaciones de datos necesarias en el mismo script.

Gestión de las dependencias y transformaciones de datos

Cuando los cambios e es del esquema requieren transformaciones de datos, versiona estos junto con los cambios del esquema. Esto garantiza que la estructura de la base de datos y sus datos permanezcan sincronizados en todos los entornos.

Basándonos en nuestro ejemplo de Flyway, las migraciones no se limitan a scripts SQL. Flyway admite migraciones basadas tanto en SQL como en Java, lo que te ofrece flexibilidad para manejar transformaciones complejas con toda la potencia de tu lenguaje de programación:

```
// V4__Add_Address_Components.java - Flyway Java-based migration
public class V4__Add_Address_Components implements JdbcMigration {

    public void migrate(Connection connection) throws Exception {
        // First, alter the schema to add new columns
        try (Statement stmt = connection.createStatement()) {
            stmt.execute("ALTER TABLE users ADD COLUMN street VARCHAR(255)");
            stmt.execute("ALTER TABLE users ADD COLUMN city VARCHAR(255)");
            stmt.execute("ALTER TABLE users ADD COLUMN state VARCHAR(255)");
            stmt.execute("ALTER TABLE users ADD COLUMN zip_code VARCHAR(10)");
            stmt.execute("ALTER TABLE users ADD COLUMN country VARCHAR(255)");
        }

        // Then, migrate the existing data
        try (PreparedStatement select = /n
        connection.prepareStatement("SELECT id, address FROM users");
             PreparedStatement update = connection.prepareStatement(
                "UPDATE users SET street = ?, city = ?, state = ?, zip_code = ?,
                country = ? WHERE id = ?")) {

            ResultSet rs = select.executeQuery();
            while (rs.next()) {
                long id = rs.getLong("id");
                String fullAddress = rs.getString("address");

                // Parse components for new schema
                AddressComponents components = addressParser.parse(fullAddress);
```

```
                    // Update user with new format
                    update.setString(1, components.getStreet());
                    update.setString(2, components.getCity());
                    update.setString(3, components.getState());
                    update.setString(4, components.getZipCode());
                    update.setString(5, components.getCountry());
                    update.setLong(6, id);
                    update.executeUpdate();
                }
            }
        }
    }
```

Este enfoque basado en código te permite aprovechar la lógica empresarial existente de tu aplicación, gestionar análisis complejos e implementar validaciones durante la migración que serían difíciles de realizar solo con SQL.

Este enfoque garantiza lo siguiente:

- El cambio de esquema y la transformación de datos se versionan juntos, manteniendo la coherencia.
- La migración es atómica: o bien se realizan con éxito tanto los cambios en el esquema como en los datos, o bien no se realiza ninguno.
- Puedes realizar un seguimiento de los entornos en los que se ha aplicado la transformación utilizando el historial de esquemas de Flyway.

Para transformaciones complejas, ten en cuenta estas opciones:

- Ejecuta las transformaciones como tareas en segundo plano para minimizar el impacto en el sistema
- Implementar un enfoque de doble escritura durante los periodos de transición
- Crear un plan de reversión en caso de que surjan problemas con la transformación

Recuerda que los problemas de calidad de los datos suelen surgir durante las transformaciones. Prepárate para gestionar datos faltantes, formatos no válidos y casos extremos que nunca habías previsto.

Cuando trabajes en transformaciones de datos, comienza con una pequeña muestra y valida los resultados minuciosamente antes de procesar todo el conjunto de datos. Este enfoque puede ahorrarte muchos dolores de cabeza.

Conclusión

Los datos son la columna vertebral de un gran porcentaje de las aplicaciones que probablemente crearás a lo largo de tu carrera profesional. Aunque puedas sentir la tentación de centrarte únicamente en los lenguajes, los marcos y las herramientas, tu capacidad para trabajar eficazmente con datos será una habilidad muy valiosa y, a menudo, determinará el éxito de tu aplicación.

A lo largo de este capítulo, hemos explorado varios aspectos del trabajo con datos:

- Comprensión de los diferentes tipos y formatos de datos (estructurados frente a no estructurados)
- Selección de los tipos de bases de datos adecuados para tus casos de uso específicos
- Implementar patrones eficaces de persistencia de datos y gestión de conexiones
- Optimizar las consultas para obtener un mejor rendimiento
- Planificar el crecimiento de los datos y las migraciones

Una conclusión e e de este capítulo es que rara vez existen soluciones perfectas en la gestión de datos, solo soluciones adecuadas para tu aplicación específica. Ninguna decisión que tomes es definitiva, y debes intentar tomar una decisión informada sobre qué estructuras de datos utilizar o qué tipo de base de datos crear y seguir adelante. Es mejor tomar una decisión basada en tus requisitos actuales que jugar al juego de «qué pasaría si» para una aplicación que podría estar disponible durante el período de tiempo o la base de usuarios que estás planificando.

Esto se denomina *sobreingeniería* y, a menudo, conduce a sistemas innecesariamente complejos que son más difíciles de mantener y adaptar a las necesidades reales. En su lugar, concéntrate en resolver los problemas actuales de forma eficaz, manteniendo tu diseño razonablemente flexible, en lugar de crear arquitecturas elaboradas para escenarios futuros hipotéticos.

Ponerlo en práctica

Trabajar con datos de forma eficaz es una habilidad que se desarrolla con la experiencia. Los conceptos que hemos tratado constituyen la base de tu viaje por el mundo de los datos. Pero, como ocurre con cualquier oficio, la maestría se consigue con la aplicación práctica.

A continuación, te ofrecemos algunas formas concretas de empezar a poner en práctica estos conceptos hoy mismo:

Analiza el modelo de datos de una aplicación existente

Toma un proyecto de código abierto en GitHub (como una plataforma de comercio electrónico sencilla o un motor de blogs) y traza un mapa de sus estructuras de datos. Identifica las relaciones entre las entidades y evalúa si el tipo de base de datos elegido se adapta a las necesidades de la aplicación.

Optimiza una consulta lenta

Busca una consulta en tu proyecto actual que no esté funcionando bien. Utiliza el comando « `EXPLAIN` » para analizar su plan de ejecución y, a continuación, implementa mejoras como añadir índices adecuados o reescribir la consulta para que sea más eficiente.

Crea una mini migración

Crea una pequeña aplicación con un esquema de base de datos básico y luego practica su evolución. Añade nuevos campos, cambia los tipos de datos o divide las tablas conservando los

datos existentes. Utiliza una herramienta de migración como Flyway o Liquibase para gestionar estos cambios.

Implementa una estrategia básica de almacenamiento en caché

Elige una función con gran volumen de lectura en una aplicación existente e implementa una capa de almacenamiento en caché sencilla. Mide el rendimiento antes y después para cuantificar la mejora.

Practica con múltiples formatos de datos

Toma un conjunto de datos en un formato (como CSV) y escribe código para transformarlo a otro formato (como JSON). Concéntrate en manejar casos extremos como caracteres especiales, valores faltantes y conversión de tipos adecuada.

Diseña una estrategia de almacenamiento para datos no estructurados

Elige un escenario (como el almacenamiento de contenido generado por los usuarios) y diseña una solución que aborde tanto el almacenamiento como la recuperación eficiente. Considera enfoques híbridos que combinen metadatos estructurados con contenido no estructurado.

Crea un prototipo de aplicación con uso intensivo de datos

Crea una pequeña aplicación que procese un gran conjunto de datos, por ejemplo, analizando archivos de registro o visualizando conjuntos de datos públicos. Céntrate en la carga eficiente de datos, la transformación y la optimización de consultas.

Empieza poco a poco a la hora de implementar nuevos conceptos de datos. ¡Lánzate hoy mismo con algo sencillo que te interese! Cada ejer-

cicio práctico refuerza tu confianza y desarrolla esa intuición especial para tomar decisiones inteligentes sobre los datos. Estos conocimientos prácticos se convertirán en uno de tus activos más valiosos a lo largo de tu carrera, te abrirán puertas y te ayudarán a resolver problemas que, de otro modo, podrían parecer abrumadores. Recuerda que todo el mundo empieza por algún sitio: tu primer proyecto de datos no tiene por qué ser perfecto, ¡solo tiene que existir!

Recursos adicionales

- *Diseño de aplicaciones con uso intensivo de datos*, 2.ª edición, de Martin Kleppmann y Chris Riccomini (O'Reilly, 2026)
- *Siete bases de datos en siete semanas*, 2.ª edición, de Luc Perkins et al. (Pragmatic Programmers, 2018)
- Dominar SQL para desarrolladores web: curso completo (en freeCodeCamp)
- *Refactoring Databases: Evolutionary Database Design*, de Scott W. Ambler y Pramod J. Sadalage (Addison-Wesley Professional, 2006)
- *Fundamentos de la ingeniería de datos*, de Joe Reis y Matt Housley (O'Reilly, 2022)
- *NoSQL destilado: una breve guía al mundo emergente de la persistencia políglota*, de Pramod J. Sadalage y Martin Fowler (Addison-Wesley Professional, 2012)

1 Un *grupo de conexiones* es una caché de conexiones a bases de datos que se mantiene en la memoria para que puedan reutilizarse cuando se requieran fu-

turas solicitudes a la base de datos, lo que elimina la sobrecarga de establecer repetidamente nuevas conexiones.